CS 263 Midterm Cheatsheet (Lectures 1–5)

Lecture 1: Intro to NLP

What NLP covers. Mapping language to structure or actions: MT, sentiment, IE, QA, instruction following, LLM generation. **Why hard.**

- **Ambiguity:** sense, syntax/PP attachment, pronoun reference.
- **Scale/drift:** huge vocabularies, long context, changing usage.
- **Reliability:** fluent output can still hallucinate.

Course arc. Task definition → metrics → representations → LMs → seq2seq/attention/transformers.

Lecture 2: Coreference Resolution

Task. Input: a document. Output: **clusters of mentions** that refer to the same entity.

- **Mentions:** pronouns, common nouns, named entities.
- Boundaries can be tricky; mentions can be nested.
- Resolution may need syntax, semantics, and commonsense.

Metrics.

- **Exact match accuracy:** 1 only if full clustering exactly matches gold.
- **Pairwise/MUC precision/recall:** compare predicted vs gold mention-pairs.
- **BCUB:** average mention-level precision/recall; often more stable than pure pairwise scoring.

$$\text{Precision} = \frac{TP}{TP+FP} \qquad \text{Recall} = \frac{TP}{TP+FN} \qquad F_1 = \frac{2PR}{P+R}$$

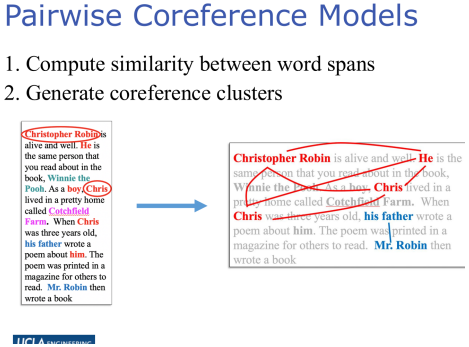
Aggregation.

- **Macro-F_1:** average document-level F_1.
- **Micro-F_1:** aggregate counts first, then compute F_1.

Modeling pipeline.

- Detect mention spans, then score span pairs.
- Useful cues: lexical match, embeddings, number/gender agreement, syntax.
- Inference: cluster by link decisions such as greedy best-left-link.
- Old: feature engineering + classifier. New: end-to-end neural mention detection + pairwise scoring.

Bias note. Coref systems can encode gender stereotypes (Wino-bias style failures); check fairness, not just average score.



Lecture 3: Word-Level Representation

One-hot vs dense.

- One-hot words are sparse, huge, and encode no similarity.
- Dense vectors let semantics appear as geometry.

Distributional hypothesis. Similar context ⇒ similar meaning. **PMI / PPMI.**

$$\text{PMI}(w_1, w_2) = \log_2 \frac{P(w_1, w_2)}{P(w_1)P(w_2)}$$

$$\text{PPMI}(w_1, w_2) = \max\left(\log_2 \frac{P(w_1, w_2)}{P(w_1)P(w_2)}, 0\right)$$

Lecture 3: Representation (cont.)

Discrete representation. One-hot vectors treat words as atomic symbols.

- Dimension $= |V|$; huge and sparse.
- No notion of similarity: dot product between different one-hots is 0.
- Unseen words/phrases get zero counts in count-based models.

Distributional hypothesis.

- Harris/Firth intuition: words occurring in similar contexts have similar meanings.
- Semantic similarity can be approximated by geometry in vector space.

PMI / PPMI.

$$\text{PMI}(w_1, w_2) = \log_2 \frac{P(w_1, w_2)}{P(w_1)P(w_2)}$$

- Positive PMI > 0: co-occur more than chance.
- Negative PMI is noisy in practice; truncate it.

$$\text{PPMI}(w_1, w_2) = \max\left(\log_2 \frac{P(w_1, w_2)}{P(w_1)P(w_2)}, 0\right)$$

Count-based embeddings.

- Build a co-occurrence matrix X from context windows/taxonomies.
- Reduce dimension with low-rank approximation / SVD.

$$X_{d\times n} \approx U_{d\times k}\Sigma_{k\times k}V^\top_{k\times n} \qquad d, n \gg k$$

Dense word vectors.

- Continuous geometry: cosine similarity, neighborhoods, analogies.
- Lecture examples: GloVe, fastText, multilingual vectors, modern embedding APIs.

Limitations.

- Co-occurrence alone can be misleading.
- Static vectors conflate word senses.
- Embeddings can encode social bias.
- Contextualized representations condition on surrounding text.

Lecture 3: Skip-gram / Word2Vec

Setup.

- Each word has two vectors: center/input v_w and outside/output u_w.
- Goal: predict context words around a center word.
- High dot product $u_o^\top v_c$ means outside word o is likely near center word c.

Softmax objective.

$$p(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w\in V}\exp(u_w^\top v_c)}$$

- Train by maximizing log-likelihood over all (center, context) pairs from a sliding window.
- Equivalent view: many multiclass classification problems; loss is cross-entropy / NLL.

Gradient intuition.

$$\frac{\partial \log p(o \mid c)}{\partial v_c} = u_o - \mathbb{E}_{w\sim p(w\mid c)}[u_w]$$

- Pull v_c toward the true context vector u_o.
- Push v_c away from expected distractor contexts.

Efficiency. Full softmax is expensive for large vocabularies; negative sampling approximates training with sampled negatives instead of summing over all words. **Static vs contextual.**

- Static embeddings give one vector per word type.
- Contextualized representations adapt to the sentence context and partially resolve sense ambiguity.

Lecture 4: Language Models

Definition. A language model scores/generates sequences:

$$P(w_{1:T}) = \prod_{t=1}^{T} P(w_t \mid w_{<t})$$

Uses. Next-word prediction, sentence scoring/ranking, generation.

n-gram LM.

- Markov assumption: only last $n-1$ tokens matter.

$$P(w_t \mid w_{<t}) \approx P(w_t \mid w_{t-n+1:t-1})$$

$$P_{\text{MLE}}(w_i \mid h) = \frac{c(h, w_i)}{c(h)}$$

- Works when counts are reliable; breaks on sparse histories.
- Zipf: a few very frequent words, long rare tail.

Smoothing.

$$P_{\text{add-1}}(w_i \mid h) = \frac{c(h, w_i)+1}{c(h)+|V|}$$

$$P_{\text{add-}\lambda}(w_i \mid h) = \frac{c(h, w_i)+\lambda}{c(h)+\lambda|V|}$$

- Reallocate mass from seen events to unseen events.
- Add-1 often over-smooths when $|V|$ is large.
- Tune λ on a dev set, not test.

Neural LM.

- Fixed-window neural LM removes sparse exact-count dependence.
- RNN LM handles variable-length contexts with shared recurrence.

$$h_t = f(W_h h_{t-1} + W_x x_t + b) \qquad p(w_t \mid h_t) = \text{softmax}(W_o h_t + b_o)$$

- Advantage: theoretically can use long context; same weights per time step.
- Failure mode: vanishing/exploding gradients.

LSTM intuition.

- Vanilla RNN keeps rewriting one hidden state.
- LSTM adds a cell state for long-term memory plus gates controlling write/erase/read.
- Helps when sequential recency and syntactic recency differ.

$$f_t = \sigma(W_f[h_{t-1}; x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}; x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}; x_t] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o[h_{t-1}; x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t)$$

Variants. Bidirectional/stacked RNNs; seq2seq composes encoder + decoder.

CS 263 Midterm Cheatsheet (Lectures 1–5)

Lecture 4/5: Seq2Seq

Encoder-decoder idea.

- Encoder maps source x to hidden states / representation.
- Decoder RNN emits target sequence autoregressively.
- Seq2seq is a **conditional** language model.

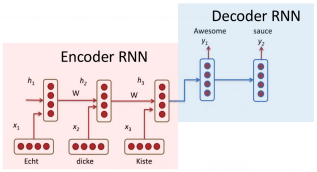
$$P(y \mid x) = \prod_{t=1}^{|y|} P(y_t \mid y_{<t}, x)$$

- Training: minimize sum of target token negative log-probabilities.
- Vanilla seq2seq bottleneck: a single fixed vector must summarize the whole source.

Good to remember.

- Encoder role: convert source sentence into useful internal states.
- Decoder role: generate target conditioned on source + previous outputs.
- Bidirectional encoders improve source coverage.

Seq2seq (Sutskever et al., 2014)



Source: http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns

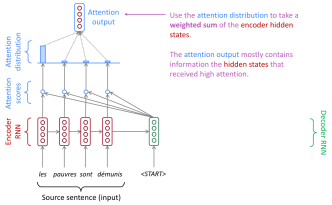
Lecture 4/5: Attention

Motivation. Avoid the bottleneck by letting each decoder step look directly at all encoder states.

$$a_{t,i} = \mathrm{sim}(h_t^d, h_i^s)$$
$$\alpha_t = \mathrm{softmax}(a_t)$$
$$c_t = \sum_i \alpha_{t,i} h_i^s$$

- Predict using $[c_t; h_t^d]$ instead of decoder state alone.
- Score function can be **multiplicative/Luong** or **additive/Bahdanau**.
- Benefits from lecture: better MT quality, bypasses bottleneck, helps gradients, provides interpretability/alignment.

Sequence-to-sequence with attention



Lecture 5: Decoding Algorithms

Search problem. Exact enumeration is exponential:

$$\text{length } K \Rightarrow O(|V|^K)$$

Greedy	Pick arg max next token each step; fast, local, brittle.
Beam	Keep top-k partial hypotheses. $k = 1$ is greedy; larger k improves search but costs more.
Sampling	Sample from $p(x_t \mid x_{<t})$ instead of taking arg max; good for open-ended generation.
Top-k	Sample only from highest-probability k tokens.
Top-p	Sample from the smallest set whose cumulative probability is at least p (nucleus sampling).

Temperature.

$$p_i(T) = \frac{\exp(s_i/T)}{\sum_j \exp(s_j/T)}$$

- $T < 1$: sharper / lower entropy.
- $T > 1$: flatter / more diverse.

Lecture 5: Transformer

Big picture. "Attention is All You Need": replace recurrence with stacked self-attention + feed-forward blocks; easier to parallelize.

Self-attention.

$$Q = HW^Q, \quad K = HW^K, \quad V = HW^V$$
$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

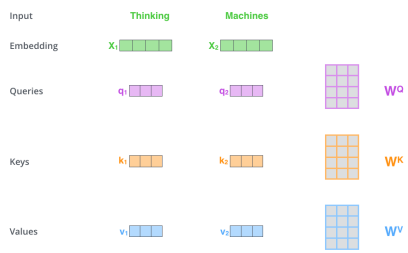
- Query decides what a position asks for.
- Key decides how relevant another position is.
- Value is the content that gets mixed in.
- Scale by $\sqrt{d_k}$ to avoid extremely sharp softmax scores.

Block structure.

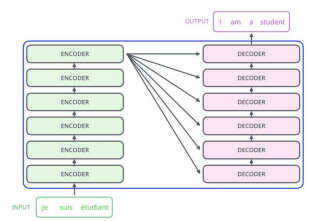
- Multi-head self-attention → feed-forward network.
- Residual connection + layer norm after each sublayer.

$$\mathrm{LayerNorm}(v) = \gamma \odot \frac{v - \mu}{\sigma} + \beta$$

Positions. Since self-attention alone is permutation-insensitive, add positional information. Lecture mentioned sinusoidal, learned position embeddings, and RoPE.



Stacks of encoders and decoders



Lecture 5: Causal Masking / Model Families

Decoder masking. Decoder-only models must not see future tokens.

- Add a lower-triangular mask to $QK^\top$ **before** softmax.
- Equivalent effect: position t can attend only to positions $\le t$.

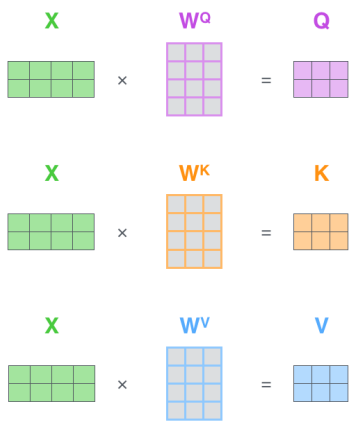
$$\begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Encoder-only vs decoder-only.

- **Encoder-only** (BERT/RoBERTa): bidirectional attention, masked language modeling.
- **Decoder-only** (GPT/LLaMA-style): masked self-attention, causal next-token prediction.

Residual connections. Needed because pure self-attention can lose self-information; skip connections preserve input features and improve optimization. **Attention matrix intuition.**

- Rows = queries, columns = keys.
- Row-wise softmax gives attention distributions.
- Visual inspection can expose alignment/coreference patterns, but attention is not a perfect explanation.



Masked Self-Attention

❖ Attention masks

What's the effect?

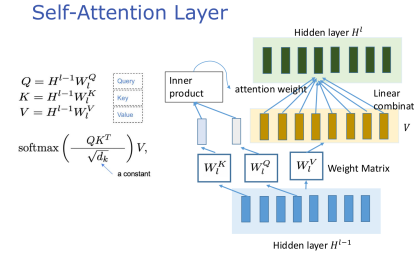
0	-inf	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf	-inf
0	0	0	-inf	-inf	-inf
0	0	0	0	-inf	-inf
0	0	0	0	0	-inf
0	0	0	0	0	0

Add to the QK^T attention BEFORE softmax.

1	0	0	0	0	0
1	1	0	0	0	0
1	1	1	0	0	0
1	1	1	1	0	0
1	1	1	1	1	0
1	1	1	1	1	1

Can we multiply this to the QK^T attention AFTER softmax?

Self-Attention Layer



https://poloclub.github.io/transformer-explainer/